第11章: Claude Codeでバイブコーディング

> ここまでは「自分の手でコードを1行ずつ書く」方法を学びました。この最終章では、AI（人工知能）と対話しながら開発を進める「**バイブコーディング**」という新しいスタイルを、**Claude Code** を例に紹介します。これまで学んだ基礎があるからこそ、AIを"正しく"使いこなせます。

---

1. バイブコーディングとは

バイブコーディング（vibe coding） とは、おおまかに言うと「**自分でコードを細かく書く代わりに、AIに『こういうものを作って』と自然言語（普通の日本語）で指示し、AIが書いたコードをレビュー・調整しながら開発を進める**」スタイルのことです。「vibe（バイブ）」は「雰囲気・ノリ」という意味の英単語で、「ノリよく、対話しながら作る」イメージです。

> **大切な前提:** バイブコーディングは「初心者がコードを理解しなくてよい」という意味ではありません。**AIの出したコードが正しいか判断し、修正を指示する**には、第1〜10章で学んだ基礎知識が不可欠です。基礎があるあなたは、AIを"優秀なアシスタント"として活用できます。逆に基礎が無いと、AIの間違いに気づけません。

1.1 なぜモバイル開発と相性が良いか

React Native開発は、覚えることが多くあります（コア部品、Flexbox、ナビゲーション、ストア提出など）。AIは「`FlatList` で本一覧を作って」「この画面をNativeWindで書き直して」といった定型的な作業を素早くこなせるため、あなたは**設計や確認といった本質的な部分に集中**できます。

---

2. Claude Code とは

Claude Code（クロード・コード） は、Anthropic社が提供するAIアシスタント「Claude」を、**ターミナル（コマンドライン）から使える開発ツール**です。プロジェクトのファイルを読み書きし、コマンドを実行し、対話しながら開発を進められます。

特徴	説明
ファイルを理解する	プロジェクト全体を読んで、文脈に合ったコードを書く
コードを書き換える	「この画面に検索を追加して」で実際にファイルを編集
コマンドを実行する	`npx expo start` やビルドコマンドを代わりに実行
対話で進む	結果を見て「もう少しこうして」と調整できる

> **他のAIツールとの違い:** ブラウザのチャット型AI（コードをコピペで往復するもの）と違い、Claude Codeは**プロジェクトのファイルを直接読み書き**します。「コピペの往復」が不要で、`my-books-app` フォルダの中で直接作業が進むのが大きな利点です。

---

3. Claude Code のセットアップ（概要）

> 本書はReact Nativeの教材なので、Claude Codeの導入は概要にとどめます。詳細・最新の手順は公式ドキュメント（https://docs.claude.com/claude-code ）を参照してください。

1. **インストール**: 公式の案内に従ってインストールします（Node.jsが前提。第1章で導入済み）。
2. **プロジェクトで起動**: ターミナルで `my-books-app` フォルダに移動し、Claude Codeを起動します。
3. **対話開始**: 「このアプリに〇〇を追加して」のように、普通の日本語で指示します。

コピー

```
cd my-books-app
# my-books-app（第1章で作ったプロジェクト）へ移動してから Claude Code を起動する
# 起動後は、プロンプトに日本語で「やりたいこと」を打ち込むだけ
```

> **プロジェクト内で起動する意味:** プロジェクトフォルダの中でClaude Codeを起動すると、AIが `app/` や `lib/` の既存コードを読んだうえで、**今のアプリに合った**変更を提案・実行してくれます。第6章で作った `lib/books.ts` の関数なども理解して使ってくれます。

---

4. 効果的な指示の出し方（プロンプトのコツ）

AIへの指示（プロンプト）の質で、結果は大きく変わります。これまで学んだ用語を使って具体的に指示するのがコツです。

4.1 良い指示・悪い指示の例

❌ あいまいな指示	⭕ 具体的な指示
「いい感じにして」	「一覧画面の本カードに、登録日（created_at）を `YYYY/MM/DD` 形式で表示して」
「バグ直して」	「保存ボタンを連打すると本が重複登録される。`saving` stateで二重送信を防いで」
「きれいにして」	「`app/new.tsx` のStyleSheetをNativeWindのclassNameに書き換えて」

> **コツ: 学んだ用語で語る:**「`FlatList`」「`useState`」「`Pressable`」「NativeWind」「RLS」といった、本書で学んだ言葉で指示すると、AIは意図を正確に汲み取ります。基礎を学んだあなたの強みです。

4.2 指示に含めると良い4要素

1. **何を**（対象ファイル・画面）:「`app/index.tsx` の…」
2. **どうしたい**（望む結果）:「…にステータスでの絞り込みを追加して」
3. **制約・方針**:「既存の検索機能と併用できるように」「NativeWindで」
4. **確認方法**:「実機で動くか `npx expo start` で確認したい」

---

5. このアプリで試せるバイブコーディングの例

第10章までででアプリは完成していますが、AIと一緒に機能拡張してみると、バイブコーディングの威力を体感できます。指示の例を挙げます。

5.1 機能追加の指示例

コピー

```
「書籍一覧に『ステータスで絞り込むタブ（すべて／未読／読書中／読了）』を追加して。
既存のキーワード検索（useMemoでのfilter）と組み合わせて動くようにして。
見た目はNativeWindで、選択中のタブを青く強調して。」
```

コピー

```
「本の詳細を見るだけの読み取り専用画面 app/books/[id]/detail.tsx を追加して。
編集画面（[id].tsx）とは別に、一覧からまず詳細を見られるようにして。
fetchBookById を使ってデータを取得して。」
```

5.2 リファクタリング（整理）の指示例

コピー

```
「app/new.tsx と app/books/[id].tsx でフォームのコードがほぼ重複している。
共通の BookForm コンポーネントを components/BookForm.tsx に切り出して、
両方の画面から使うようにリファクタリングして。」
```

> 「**リファクタリング**」とは？ 動作を変えずに、コードの構造を整理・改善することです。重複したコードを共通部品にまとめる、といった作業はAIが得意とするところです。第3章で学んだ「コンポーネントで再利用」の考え方が、ここで活きます。

5.3 エラー解決の指示例

コピー

```
「npx expo start を実行したら次のエラーが出た。原因と修正方法を教えて。
（エラーメッセージをそのまま貼り付ける）」
```

> **エラーは"そのまま貼る"のが鉄則:** エラーメッセージは、原因を特定する最重要の手がかりです。要約せず、ターミナルに出た文面を**そのままコピーして貼る**と、AIは的確に診断できます。

---

6. AIを使うときの注意点

バイブコーディングは強力ですが、**鵜呑みは禁物**です。以下を必ず意識してください。

6.1 出力されたコードは必ず確認する

AIは時々、間違ったコードや、古い書き方、存在しない部品を提案することがあります（これを「ハルシネーション＝もっともらしい嘘」と呼びます）。**生成されたコードは必ず自分の目で確認し、実機で動作確認**しましょう。本書で基礎を学んだあなたなら、おかしな点に気づけます。

6.2 セキュリティ情報を渡さない

`.env` の中身（Supabaseの `service_role` キーなど秘密情報）や、ユーザーの個人情報を不用意にAIに貼らないようにします。`anon` キーは公開前提なので問題ありませんが、**秘密キーは扱いに注意**です（第5章参照）。

6.3 大きな変更は小さく分ける

「アプリ全体を作り直して」のような巨大な指示より、「まずこの画面だけ」「次にこの機能」と**小さく区切って**進める方が、確認しやすく失敗が少ないです。第6章以降で1章ずつ機能を積み上げたのと同じ考え方です。

6.4 Gitでこまめに記録する

AIの変更を試す前に、`git commit`（第1章）で現状を記録しておくと、「AIの変更で壊れた」ときにすぐ元へ戻せます。

コピー

```
git add .
# git add .   : 変更したファイルすべてを「記録対象」に登録（.は今のフォルダ全体）

git commit -m "検索機能まで実装した時点"
# git commit  : 現在の状態をスナップショットとして記録
# -m "..."    : コミットメッセージ（記録の説明）。後で見て分かる内容にする
```

> **なぜGit記録が大事？** AIに大きな変更を任せた結果、うまく動かなくなることがあります。直前にコミットしておけば `git` で元の状態に戻せるので、安心して試行錯誤できます。「実験の前にセーブする」感覚です。

---

7. これからの学習の進め方

バイブコーディングを取り入れつつ、スキルを伸ばしていくための指針です。

レベル	おすすめの進め方
今（基礎習得直後）	AIに頼りすぎず、まず自分で書いてみる。詰まったらAIに聞く
慣れてきたら	定型作業はAIに任せ、設計・確認に集中する
さらに上達	AIの提案を批評・改善できるようになる。「なぜこの書き方か」を考える

> **AIは「先生」でもある:** Claude Codeに「このコードはなぜこう書くの？」「`useMemo` と `useEffect` の使い分けは？」と質問すれば、丁寧に解説してくれます。**分からないことをその場で聞ける家庭教師**として使うのも、とても効果的な学び方です。

---

8. この章のまとめ

- **バイブコーディング** = AIと対話しながら開発するスタイル。**基礎があってこそ**活用できる
- **Claude Code** はターミナルからプロジェクトを直接読み書きするAI開発ツール
- 指示は **具体的に・学んだ用語で・小さく区切って** 出すのがコツ
- 生成コードは**必ず確認・実機テスト**。秘密情報は渡さない。Gitでこまめに記録する
- AIは「優秀なアシスタント」かつ「いつでも聞ける先生」として活用できる

---

9. 全課程の修了 — 本当におめでとうございます！ 🎓

この教材で、あなたは次を成し遂げました。

- 📱 **ゼロからスマホアプリを構築**（TypeScript / React / React Native / Expo）
- 🗄️ **データベースと連携**（Supabaseで CRUD）
- 🎨 **見た目を整える**（NativeWind）
- 🚀 **アプリストアに公開**（EAS）
- 🤖 **AIと協働する**（Claude Codeでバイブコーディング）

これは、現代のアプリ開発者が持つべきスキルの「ひと通り」です。ここからは、自分のアイデアを形にしていく番です。作りたいものを作り、世界に届けてください。あなたの開発の旅が、ここから始まります。お疲れさまでした！ 🎉

> **姉妹編のご案内:** 同じ「書籍管理アプリ」をWeb（ブラウザ）向けに作る `next` フォルダの教材もあります。**同じSupabaseバックエンドを共有**できるので、「Web版とモバイル版を両方持つ」マルチプラットフォーム開発に挑戦するのも、素晴らしい次の一歩です。